

TECHNICAL SOLUTION DOCUMENT - REACT NATIVE APP

Submitted By:

Rajath Kumar

1rajathkumar@gmail.com

7022167082

TABLE OF CONTENT

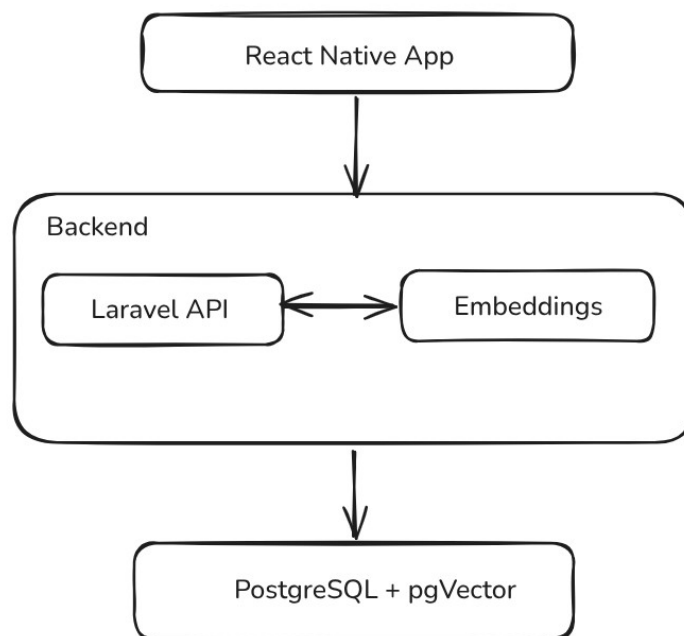
Sl. No.	Title	Page No
1.	Introduction	3
2.	High Level Architecture	3
3.	Database Schema Design	3-4
4.	Vector Embedding	4
5.	API Design	4-7
6.	Feed Ranking Algorithm	8
7.	Design Choices and Trade-offs	9
8.	AI Agentic Tool	9-10
9	Reference	10

1. INTRODUCTION

The objective of this project is to design and implement the **Real Connections Feed** which unlike traditional social media feeds that prioritize engagement metrics such as likes or comments. Feed ranking content using meaningful social interactions, semantic similarity and authenticity while maintaining real-time responsiveness.

In this document explains the architectural decisions, database design, ranking algorithm, API design and implementation strategy before development begins.

2. HIGH LEVEL ARCHITECTURE



The backend system is designed using Laravel and Python. Laravel acts as the primary API layer also works on authentication, migrations, routing and testing. While the python handles the sentence transformation and embeddings.

The database layer is designed using PostgreSQL and pgvector. PostgreSQL is resilient, ACID compliant and works well with Laravel while pgvector is simple easy to use and works without any unnecessary complexity with postgresQL.

3. DATABASE SCHEMA DESIGN

The overall relationship is illustrated in the ER diagram shown below.

- A **User** can create multiple **Posts**.

- Every **Post** has exactly one **Embedding** used for semantic search.
- A **User** can perform multiple **Interactions** on different posts.
- Each **Interaction** belongs to both a user and a post.

Contains 4 tables – users, posts, interactions and post embeddings

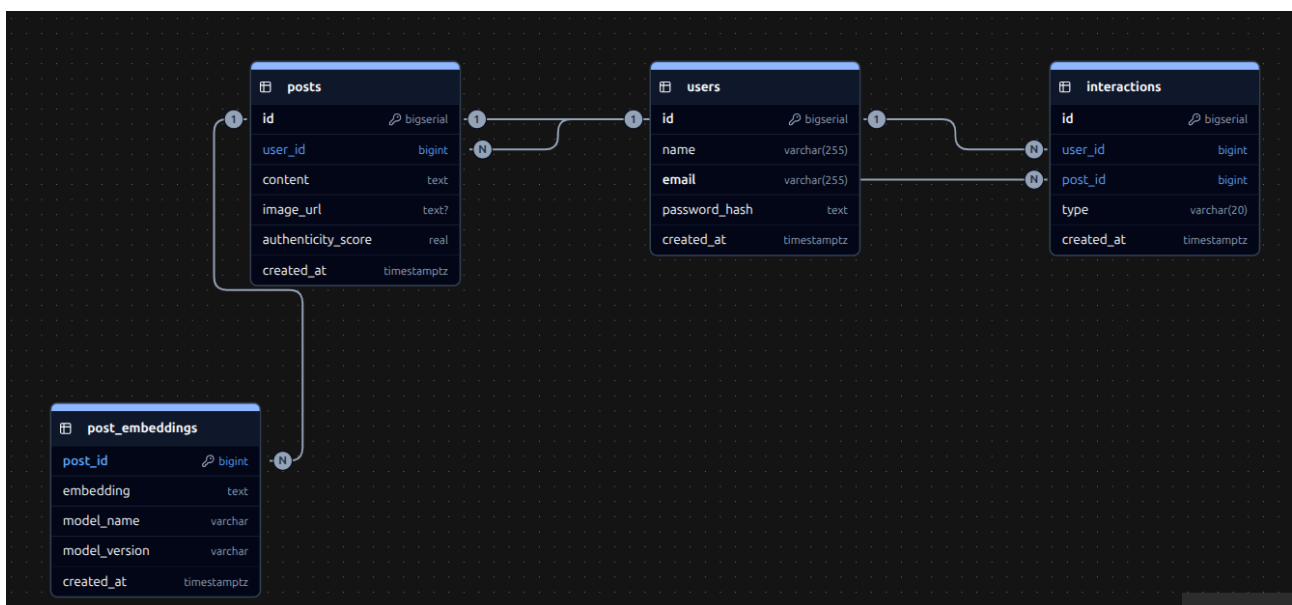
1. users - Stores authentication and profile information for every registered user.

2. posts - Each post belongs to exactly one user through a foreign key relationship. A post contains textual content, an optional image URL, an authenticity score, and metadata such as creation time.

3. post_embeddings - Stores the vector representation of every post. Instead of storing embeddings inside the Posts table, they are isolated into a dedicated table. This separation improves maintainability and allows the embedding model to evolve without modifying the business data.

4. interactions - Records every meaningful user action performed on a post.

Each interaction links one user to one post and records the interaction type (view, reply or reaction) along with the timestamp.



4. VECTOR EMBEDDING

As for the embedding model we will be using sentence-transformers. **all-MiniLM-L6-v2**, run inside the Python service, no API key required. Every post is converted into an embedding when it is created. Likewise, when a user performs a natural language search, the search query is converted into an embedding, and a similarity search is performed against stored post embeddings.

Other choices being OpenAI Embeddings are great for such tasks but requires API credits and additional latency cost.

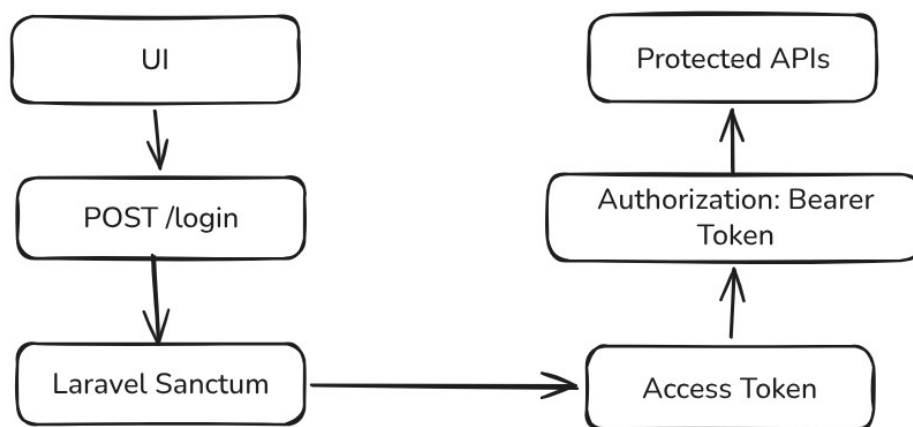
5. API DESIGN

The backend follows a RESTful architecture where resources are represented as endpoints and standard HTTP methods are used to perform operations. The API is stateless, meaning every request contains the necessary authentication information and no session state is stored on the server.

The project uses **Laravel Sanctum** for token-based authentication, as required by the assignment.

After successful login, Laravel issues a personal access token. The client stores this token securely and includes it in every subsequent request using the Authorization header.

API FLOW



Pass the authentication token on every request

Authorization: Bearer <access_token>

Method	Endpoint	Description
POST	/api/posts	Create a new post.
GET	/api/feed	Return a personalized feed for the authenticated user.
GET	/api/search?q=...	Natural language search across posts using vector similarity.
POST	/api/interactions	Log a user interaction (view, reply, reaction) against a post.

Http Status response:

200 – Ok

201 – Resource Created

400 – Validation Failed

401 – Unauthenticated

500 – Server Error

1. POST /api/posts

Creates a new post and automatically generates an embedding for semantic search.

Request:

```
</> JSON

{
  "content": "Had an amazing trip to Goa this weekend!",
  "image_url": "https://cdn.app/images/goa.jpg"
}
```

Response:

```
</> JSON

{
  "message": "Post created successfully",
  "data": {
    "id": 21,
    "content": "Had an amazing trip to Goa this weekend!",
    "image_url": "https://cdn.app/images/goa.jpg",
    "authenticity_score": 0.83,
    "created_at": "2026-06-27T10:15:00Z"
  }
}
```

2. GET /api/feed

Returns a paginated personalized feed for authenticated users. Page number is defaulted to 1

Optional parameter → /api/feed/page=2

Response:

```
</> JSON

{
  "current_page": 1,
  "has_next_page": true,
  "data": [
    {
      "post_id": 15,
      "username": "Alice",
      "content": "Morning trek!",
      "image_url": null,
      "authenticity_score": 0.91,
      "score": 0.87,
      "created_at": "2026-06-27T09:00:00Z"
    }
  ]
}
```

3. GET /api/search?q=funny travel stories

Allows users to search using natural language.

Optional parameter → GET /api/search?q=funny travel stories&page=1

Request:

Pass a query parameter, q

Response:

```
JSON
{
  "query": "funny travel stories",
  "results": [
    {
      "post_id": 9,
      "username": "John",
      "similarity": 0.94,
      "content": "Missed my train while backpacking..."
    }
  ]
}
```

4. POST /api/interactions

Stores user interactions that contribute to relationship depth.

Interface type → view, reply, reaction

Request:

```
JSON
{
  "post_id": 15,
  "type": "reaction"
}
```

Response:

```
JSON
{
  "message": "Interaction recorded successfully."
}
```

6. FEED RANKING ALGORITHM

The goal of this is to make sure that content stays relevant to the customers using the site not only based on the number of likes but on the basis of relevance.

Each candidate post gets scored on three things, then the whole score is discounted by age:

1. **Semantic similarity** — how close the post's embedding is to a vector representing what this user cares about (a rolling average of embeddings from posts they've recently engaged with).
2. **Relationship depth** — a normalized count of interactions (view/reply/reaction, weighted differently) between this user and the post's author over the last 30 days.
3. **Authenticity** — a stored heuristic score, penalize heavy hashtag use and very short captions, slightly reward first-person/conversational text. Each post receives an authenticity score ranging from 0 to 1.

This score is then multiplied by an exponential time-decay factor to make sure that older content that are highly relevant still ranks above a new less relevant content.

Weighted Ranking formula:

Signal	Interaction	Score
Relationship depth (40%)	Reply	3.0
	Reaction	2.0
	View	1.0
Semantic Similarity (30%)	Based on embeddings	0 - 1
Authenticity Score (20%)	Content Authenticity	0-1
Time Decay (10%)	Post relevance	$e^{(-\lambda \times \text{age})}$

Note: λ is a hyperparameter

Pseudo Code:

```
</> pseudo
candidatePosts = getRecentPosts()

for each post in candidatePosts

    relationshipScore =
        calculateRelationship(currentUser, post.author)

    semanticScore =
        cosineSimilarity(
            currentUser.embedding,
            post.embedding
        )

    authenticityScore =
        post.authenticityScore

    timeScore =
        exponentialDecay(post.createdAt)

    finalScore =
        0.40 * relationshipScore
        + 0.30 * semanticScore
        + 0.20 * authenticityScore
        + 0.10 * timeScore

sort posts by finalScore descending

return first 20 posts
```


7. DESIGN CHOICE AND TRADE-OFFS

1. PostgreSQL + pgvector vs Dedicated Vector Database

Decision: PostgreSQL with pgvector

Why: Integration between pgvector and postgresSQL is seamless without any additional overhead.

Alternative: Qdrant or Pinecone

Better scaling, higher throughput, production grade features with distributed indexing.

2. Sentence Transformers vs OpenAI Embeddings

Decision: Sentence Transformers

Why: Open-source and free, Faster development, No external dependency and no additional network latency.

Alternative: text-embedding-3-small

Higher accuracy in semantic embedding and multilingual support

3. Weighted Ranking vs Machine Learning Ranking Model

Decision: Weighted scoring algorithm.

Why: We do not store the historical interactions for training a ml model for better recommendation. Manual weighted ranking is clean, simple and easy to tune

Alternative: XGBoost, LightGBM Ranker, Neural Ranking Models

As user base grows and interactions increases ml models can learn from pattern and rank feed better than weighted averages.

4. Synchronous Embedding Generation vs Asynchronous Processing

Decision: Synchronous generation during post creation.

Why: Immediately generating and storing embeddings ensures every newly created post is searchable without requiring additional infrastructure.

Alternative: RabbitMQ, Kafka, AWS SQS

A queue based architecture that works asynchronously is more scalable and takes less time during the write operation of a database.

8. AI AGENTIC TOOLS

The entire project was built with the help of AI tools primarily ChatGPT.

Link to the conversations with ChatGPT

ChatGPT → <https://chatgpt.com/share/6a3eb210-4c68-83e8-a031-4baedb0c2a31>

TSD:

Page No: 9

Used AI for understanding the project requirement, understand the trade-offs and help with the content for the creation of this document. All generated code and architectural suggestions were reviewed, modified where necessary, and validated before being incorporated into the project.

BACKEND:

Modularized built with the help of AI prompts with more emphasis to understanding by comparing already known backend framework (SpringBoot and Node.js) with Laravel.

EMBEDDING SERVICE:

MicroService architecture guide, helped in integrating existing backend with python FastAPI backend to make use of sentence formation.

REACT NATIVE APP:

Help with the folder structure, decision making and entire code prompts with debugging.

9. REFERENCE:

GitHub	https://github.com/Rajathk6/GuishedUp
Postman	https://www.postman.com/db-performance/workspace/guishedup
ChatGPT	https://chatgpt.com/share/6a41639a-b678-83ee-85e1-9f3891b96903
Drive	https://drive.google.com/drive/folders/1g5GHUb5SQgeUtyVtFFIWq92AaIzAp6pz?usp=sharing